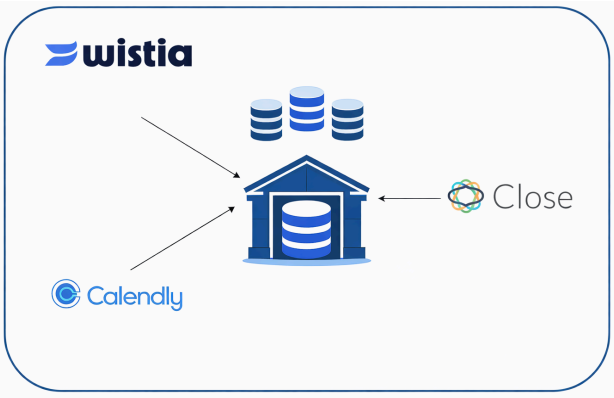


Business Context:

Modern revenue teams rely on multiple tools to engage prospects, schedule conversations, and convert leads into customers. However, data from these systems often remains siloed, limiting visibility into the full customer journey and slowing down sales operations.

This project simulates a real-world, end-to-end data engineering initiative where marketing engagement data (Wistia), meeting scheduling data (Calendly), and CRM lead data (Close) are integrated into a unified data platform. The goal is to create a single source of truth that supports both **business analytics** and **operational automation**.

Objectives:

The unified platform aims to:

- Capture **video engagement analytics** from Wistia (media-level and visitor-level) across marketing channels such as Facebook and YouTube
- Ingest **meeting scheduling data** from Calendly to analyze lead behavior, meeting trends, and calendar utilization
- Consume **real-time CRM events** from Close via webhooks to track new lead creation and ownership updates
- Centralize all data into a **cloud-based data warehouse** for analytics and reporting
- Deliver **real-time notifications** to sales teams with enriched lead and engagement context
- Generate insights/visualizations to optimize **marketing spend, conversion rates, and sales response time**

Technical & Business Requirements:

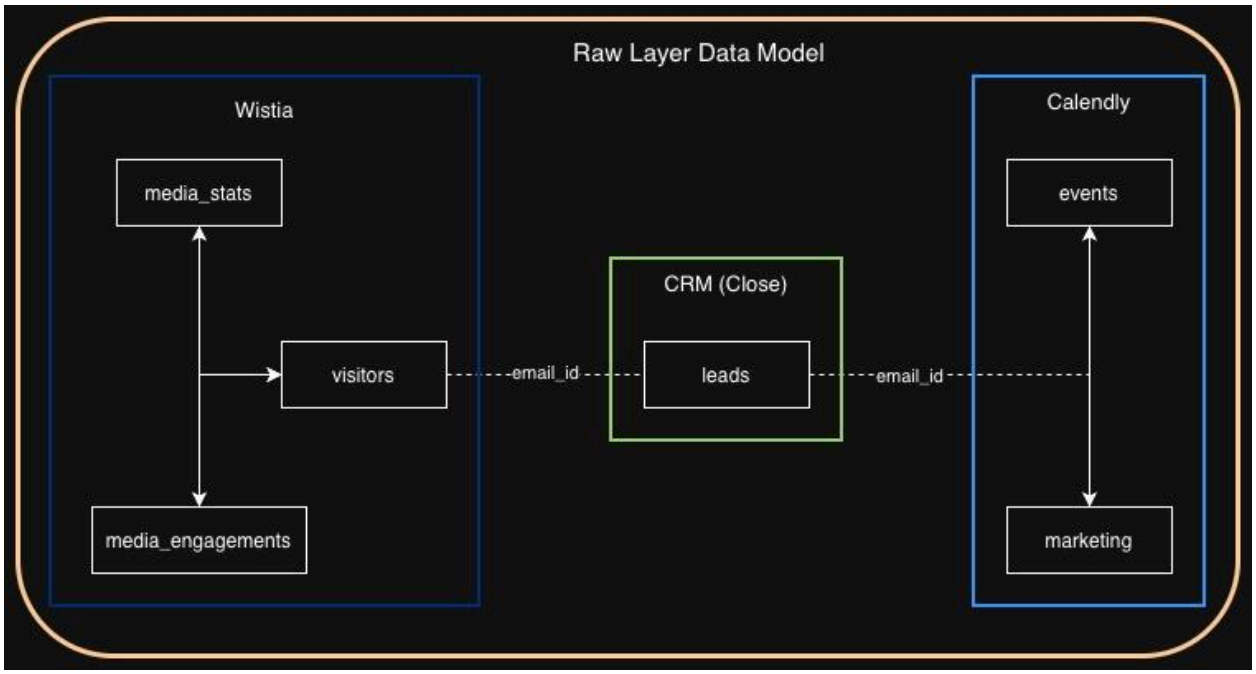
- **Data Sources:**
  - CRM (Close) (Webhook subscription)
  - Calendly (Webhook subscription)
  - Wistia video analytics (API)

- **Data Format:** JSON
- **Real Time Feeds:** CRM & Calendly events
- **Batch Feeds:** Wistia data
- **Real Time Transformation:** CRM
- **Batch Transformation:** Calendly & Wistia

System Design:

- The complete **end-to-end architecture** must be **designed and owned by the project team**.
- DO NOT USE DBT FOR TRANSFORMATION.
- DO NOT USE ANYTHING APART FROM AWS/Azure.
- The end-to-end data pipeline for all three sources must be **operational and scheduled to run for 7 consecutive days**.
- The system should demonstrate:
    - Reliable ingestion
    - Idempotent processing
    - Fault tolerance and recovery handling
- You cannot use ***Snowflake***
- Prepare and present the **system architecture and data model** for **business and technical approval** by SMEs (**Ninad, Azmat, Kareem, Avirup**).
- Proceed with implementation **only after receiving formal approval** from the SMEs.

Data Model:



ALL THE REQUIREMENTS FOR INDIVIDUAL FLOW ARE GIVEN BELOW

CRM Lead Data Pipeline Requirements

Step 1: Source Data Explanation:

- Go to Close Webhook Subscription Documentation : *https://developer.close.com/resources/webhook-subscriptions/#create-new-webhook-subscription*
- You must use the left-side pane to explore the other routes of the API.

Step 2: Initial Data Analysis:

- Investigate the sample webhook event -

{'resource': '/crm', 'path': '/crm', 'httpMethod': 'POST', 'headers': {'Accept': '*/*', 'Accept-Encoding': 'gzip, deflate', 'baggage': 'sentry-trace_id=61a24e3effec455dbbdd7e64cd92ed34,sentry-environment=production,sentry-release=8cc593e4c43631cc,sentry-public_key=244de8807ed34db163a207e1fa1be6e6', 'Close-Sig-Hash': '039d2e5f719c11a63830cf68af480a3eae1f56fb7f39c057927ed68dc432c33d', 'Close-Sig-Timestamp': '1756970837', 'content-type': 'application/json', 'Host': '280utl7289.execute-api.us-east-1.amazonaws.com', 'sentry-trace': '61a24e3effec455dbbdd7e64cd92ed34-9b19a5131ec8a33a', 'User-Agent': 'Close Webhooks 2.0', 'X-Amzn-Trace-Id': 'Root=1-68b93f56-68539cc770cbfba028c12808', 'X-Forwarded-For': '54.186.6.46', 'X-Forwarded-Port': '443', 'X-Forwarded-Proto': 'https'}, 'multiValueHeaders': {'Accept': ['*/*'], 'Accept-Encoding': ['gzip, deflate'], 'baggage': ['sentry-trace_id=61a24e3effec455dbbdd7e64cd92ed34,sentry-environment=production,sentry-release=8cc593e4c43631cc,sentry-public_key=244de8807ed34db163a207e1fa1be6e6'], 'Close-Sig-Hash': ['039d2e5f719c11a63830cf68af480a3eae1f56fb7f39c057927ed68dc432c33d'], 'Close-Sig-Timestamp': ['1756970837'], 'content-type': ['application/json'], 'Host': ['280utl7289.execute-api.us-east-1.amazonaws.com'], 'sentry-trace': ['61a24e3effec455dbbdd7e64cd92ed34-9b19a5131ec8a33a'], 'User-Agent': ['Close Webhooks 2.0'], 'X-Amzn-Trace-Id': ['Root=1-68b93f56-68539cc770cbfba028c12808'], 'X-Forwarded-For': ['54.186.6.46'], 'X-Forwarded-Port': ['443'], 'X-Forwarded-Proto': ['https']}, 'queryStringParameters': None, 'multiValueQueryStringParameters': None, 'pathParameters': None, 'stageVariables': None, 'requestContext': {'resourceId': 'kqvc6w', 'resourcePath': '/crm', 'httpMethod': 'POST', 'extendedRequestId': 'QXbVgFqQIAMEZkw=', 'requestTime': '04/Sep/2025:07:27:18 +0000', 'path': '/deploy/crm', 'accountId': '294845879996', 'protocol': 'HTTP/1.1', 'stage': 'deploy', 'domainPrefix': '280utl7289', 'requestTimeEpoch': 1756970838008, 'requestId': '61ccfc0e-5caa-48e0-acf2-b94a6eb2a814', 'identity': {'cognitoIdentityPoolId': None, 'accountId': None, 'cognitoIdentityId': None, 'caller': None, 'sourceIp': '54.186.6.46', 'principalOrgId': None, 'accessKey': None, 'cognitoAuthenticationType': None, 'cognitoAuthenticationProvider': None, 'userArn': None, 'userAgent': 'Close Webhooks 2.0', 'user': None}, 'domainName': '280utl7289.execute-api.us-east-1.amazonaws.com', 'deploymentId': 't2emw1', 'apiId': '280utl7289'}, 'body': '{"subscription_id": "whsub_1Lr0LeQ56WDreQtJNiuUAr", "event": {"id": "ev_70750rgrOVLfmED9y7ExRy", "date_created": "2025-09-04T07:27:17.674000", "date_updated": "2025-09-04T07:27:17.674000", "organization_id": "orga_rq7EW7TTwnrAshSzIQ49jJUWzDxtTQUBBrA4VoRWEID", "user_id": "user_RoWq89U2oZOlTTfJScR9pTREnX3AAeNdKnw4YOOcYwG", "request_id": "req_6wqaiW5aRzHH0U8jAg8jJI", "api_key_id": null, "oauth_client_id": "oa2client_0RZ3Soibz8n7hqA2D6bmlV", "oauth_scope": "all.full_access offline_access", "object_type": "lead", "object_id": "lead_Q6uqUbqVS4NYyHGJnfxpfNKpZ0roxEDCdD9oIAaMJ4z", "lead_id": "lead_Q6uqUbqVS4NYyHGJnfxpfNKpZ0roxEDCdD9oIAaMJ4z", "action": "created", "changed_fields": [], "meta": {"request_path": "/api/v1/lead/", "request_method": "POST"}, "data": {"date_updated": "2025-09-04T07:27:17.672000+00:00", "addresses": [], "organization_id": "orga_rq7EW7TTwnrAshSzIQ49jJUWzDxtTQUBBrA4VoRWEID", "updated_by": "user_RoWq89U2oZOlTTfJScR9pTREnX3AAeNdKnw4YOOcYwG", "display_name": "Adam Capozzoli", "updated_by_name": "Christopher Garzon", "contact_ids": ["cont_VFH15lJgy9GxEvRNK3VcuZnxR24hb4PJo0COMVKdZ0Q"], "id": "lead_Q6uqUbqVS4NYyHGJnfxpfNKpZ0roxEDCdD9oIAaMJ4z", "name": "", "created_by_name": "Christopher Garzon", "created_by": "user_RoWq89U2oZOlTTfJScR9pTREnX3AAeNdKnw4YOOcYwG", "url": null, "date_created": "2025-09-04T07:27:17.642000+00:00", "status_label": "Potential", "description": "", "status_id": "stat_w883GlAbRCWhns1gvURwAaImLfsmA4SwN5HYLsToMOY", "custom.cf_am3UgCUhyM5iNDtAPL84enDjUrZx1JsyVZ9uD9TbYwG": "YT DE ACADEMY Direct VSL", "custom.cf_DeLLGkRULjyldwA4CA5umg5YrhOT6y0Ex44eycTadS2": "13386968", "custom.cf_7ZWtXzOYPfLduhXb8fCM0m6L2KawUuubVHq12dKgvI9": "_____", "custom.cf_jr2d8YqWeVF2GF8UqYTLjnOhCF9N61vUOnTTHrwPcPz": "62330063", "custom.cf_rCyWUZqqs1zp8gS8fk5GTPFgzLBCt6WQhmrSn9tnJMo": "Arizona"}, "previous_data": {}}}', 'isBase64Encoded': False}

- Investigate the sample lead owner file -

{
"lead_id": "lead_niuYPXlw6vnFQIhwZCKDaaKv9XMQs9KA5NgxhNRBgaA",
"display_name": "Lowell Bast",
"lead_email": "gatequdit@gmail.com",

```
"lead_owner": "Lucija Bitunjac",
"funnel": "DE ACADEMY Direct VSL",
"status_label": "Potential",
"date_created": "2025-05-20T12:14:56.409000+00:00"
}
```


- Sample event data from the CRM Webhook Stored in S3 Bucket -


```
{"subscription_id": "whsub_1zwvQVQOBPFqrH6xXJEvFJ",
"event": {
"id": "ev_1ntH1vAE4G7DNjNZYkMeck",
"date_created": "2025-05-20T12:14:56.446000",
"date_updated": "2025-05-20T12:14:56.446000",
"organization_id": "orga_rq7EW7TTwnrAshSzlQ49jJUWzDxtTQUBBrA4VoRWElD", "user_id":
"user_RoWq89U2oZOlTTfJScR9pTREnX3AAeNdKnw4YOOcYwG",
"request_id": "req_1B0t9wF68LfveKOdLNsbVE",
"api_key_id": null,
"oauth_client_id": "oa2client_0RZ3Soibz8n7hqA2D6bmlV",
"oauth_scope": "all.full_access offline_access",
"object_type": "lead", "object_id": "lead_niuYPXlw6vnFQIhwZCKDaaKv9XMQs9KA5NgxhNRBgaA",
"lead_id": "lead_niuYPXlw6vnFQIhwZCKDaaKv9XMQs9KA5NgxhNRBgaA",
"action": "created",
"changed_fields": [],
"meta": {
"request_path": "/api/v1/lead/",
"request_method": "POST"
},
"data": {
"status_id": "stat_w883GIAbRCWhns1gvURwAaImLfsmA4SwN5HYLsToMOY", "display_name": "Lowell
Bast",
"created_by": "user_RoWq89U2oZOlTTfJScR9pTREnX3AAeNdKnw4YOOcYwG", "created_by_name":
"Christopher Garzon",
"addresses": [],
"name": "",
"description": "",
"url": null,
"contact_ids": ["cont_xASn6XuY8qQuUq6FN919ia9jjg3QvkKXDXx90pytjAE"], "date_updated":
"2025-05-20T12:14:56.444000+00:00",
"organization_id": "orga_rq7EW7TTwnrAshSzlQ49jJUWzDxtTQUBBrA4VoRWElD", "updated_by":
"user_RoWq89U2oZOlTTfJScR9pTREnX3AAeNdKnw4YOOcYwG",
"id": "lead_niuYPXlw6vnFQIhwZCKDaaKv9XMQs9KA5NgxhNRBgaA",
"updated_by_name": "Christopher Garzon",
"status_label": "Potential",
"date_created": "2025-05-20T12:14:56.409000+00:00",
"custom.cf_am3UgCUhyM5iNDtAPL84enDjUrZx1JsyVZ9uD9TbYwG": "DE ACADEMY Direct VSL",
"custom.cf_DeLLGkRULjyldwA4CA5umg5YrhOT6y0Ex44eycTadS2": "13074380",
"custom.cf_jr2d8YqWeVF2GF8UqYTLjnOhCF9N61vUOnTTHrwPcPz": "63042928",
"custom.cf_rCyWUZqqs1zp8gS8fk5GTPFgzLBCt6WQhmrSn9tnJMo": "Eastern Time (US & Canada)"
},
"previous_data": {}
}
}
```


Step 3: Build the Data Pipeline:

1. API with **POST** method needs to be created: You need to send the API URL to **SME(Avirup/Azmat/Ninad/Kareem)** for Webhook creation.
2. **Store webhook data in JSON format in Amazon S3 bucket source folder after processing the required details from the webhook event.**

   The name of the file should be as (**crm_event_{lead_id}.json**) - *lead_id can be fetched from the CRM event.*

3. **Implement a 10-minute delay mechanism to provide sufficient time for the CRM system to assign a lead owner before processing the lead further.** For reference :

Introducing Delay -
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-delay-queues.html

For S3 Event Notification -
https://docs.aws.amazon.com/AmazonS3/latest/userguide/ways-to-add-notification-config-to-bucket.html#step1-create-sqs-queue-for-notification

4. **Lookup recently updated lead owner files in another S3 location. Merge lead and lead owner data using lead_id and store data into the S3 bucket target folders**

   Use the following public S3 bucket to perform a lookup and assign lead owner details to each newly created lead that was stored in S3 from the CRM webhook event.

   Public S3 Bucket and File Details

   bucket_name : dea-lead-owner

   file_name = f"{lead_id}.json"

   public_url = f"https://{bucket_name}.s3.us-east-1.amazonaws.com/{file_name}"

5. **Send notification with relevant data to the team as notification either on Slack or in an email**

   Make sure that the below fields are available as part of the notification sent to either as slack notification or email notification

   New Lead Alert

   Name: {display_name} - Fetch from the CRM Event data

   Lead ID : {lead_id} - Fetch from the CRM Event data

   Created Date: {date_created} - Fetch from the CRM Event data

   Label: {status_label} - Fetch from the CRM Event data

   Email: {lead_email} - Fetch from the lookup data

   Lead Owner: {lead_owner} - Fetch from the lookup data

   Funnel: {funnel} - Fetch from the lookup data

   ***Reference** : Sending messages using incoming webhooks in Slack*

   https://api.slack.com/messaging/webhooks

Step 4: Outcomes:

- Leads are correctly ingested and saved in S3 within seconds.
- Lead owner assignment is done only after the delay (not before).
- Lookup and enrichment match correctly by lead_id.
- The team receives real-time notifications with accurate data.
- The system handles multiple leads in parallel without race conditions.
- Create **final reports or dashboards** which consolidate the CRM, Wistia & Calendly data to find relevance between the marketing efforts, video visitors analysis and the new lead generation.
- Please go through this link for working on Streamlit for Visualization: Streamlit Tutorial

Wistia Data Flow Requirements

Step 1: Source Data Explanation:

- Go to Wistia API Documentation: https://docs.wistia.com/reference/get_stats-medias-mediaid
- On the left side pane, under **Stats API** find different types of datasets for stats.

Step 2: Configuration Details:

- API Token (DO NOT SHARE) to be used**:** 0323ade64e13f79821bdc0f2a9410d9ec3873aa9df01f8a4a54d4e0f3dd2e6b4
- **Media Ids** to be used: **8hunphufxp & 9k4tbcdfg0**

Step 3: Build the Data Pipeline:

- Authenticate to **Wistia Stats API** using token-based Basic Auth
- Extract **media metadata** (title, ID, hashed_id, created_at, etc.)
- Extract **engagement metrics** (plays, play rate, watch time, etc.)
- Extract **visitor-level data** (IP, engagement events)
- Implement **pagination** to fetch all pages of results
- Implement **incremental ingestion** based on created_at/updated_at

- Create **final reports or dashboards** which consolidate the CRM, Wistia & Calendly data to find relevance between the marketing efforts, video visitor analysis and the new lead generation.
- Please go through this link for working on Streamlit for Visualization: Streamlit Tutorial

Calendly Data Flow Requirements

Step 1: Source Data Explanation:

- Go to Calendly - Developer Website
- On the left side pane, under **Resources** find different types of datasets in json format namely ***scheduled events & webhooks***.
- **Calendly data** will be **received using webhooks** that will have event related information which are getting newly created.

Step 2: Initial Data Analysis:

- ○ Analyze the json datasets of ***webhooks*** and determine the input needed.(**Manually**)
- ○ Identify the json data elements to determine the table structure needed for designing your **Bronze, Silver & Gold** layer of calendly data.(**Manually**)
- ○ Data Hierarchy and Identification Process:
    - API with **POST** method: You need to send the API URL to SME(Avirup/Azmat/Ninad) for Webhook creation.

Organization_URL - https://api.calendly.com/organizations/8910725e-6409-44dc-ba57-ead6dd48b052

- **Sample Incoming Webhook Events**:

Webhook JSON for invitee.created**:**

{"created_at":"2025-07-09T06:04:34.000000Z","created_by":"https://api.calendly.com/users/8ec9d4df-39c5-463a-8b40-bcb5d0a70edf","event":"invitee.created","payload":{"cancel_url":"https://calendly.com/cancellations/22a0f2d6-1bde-4fc1-95c1-d969df1da21d","created_at":"2025-07-09T06:04:33.761871Z","email":"chrisgarzon19@gmail.com","event":"https://api.calendly.com/scheduled_events/1ac9e88e-eae3-4e4b-b979-d770cff02d72","first_name":null,"invitee_scheduled_by":"https://api.calendly.com/users/8ec9d4df-39c5-463a-8b40-bcb5d0a70edf","last_name":null,"name":"Ninad Magdum","new_invitee":null,"no_show":null,"old_invitee":null,"payment":null,"questions_and_answers":[{"answer":"+91 91303 02575","position":0,"question":"What is your phone number?"}],"reconfirmation":null,"reschedule_url":"https://calendly.com/reschedulings/22a0f2d6-1bde-4fc1-95c1-d969df1da21d","rescheduled":false,"routing_form_submission":null,"scheduled_event":{"created_at":"2025-07-09T06:04:33.743275Z","end_time":"2025-07-09T18:00:00.000000Z","event_guests":[],"event_memberships":[{"user":"https://api.calendly.com/users/76e8f2b2-b38c-41c8-826f-4b61f2ba22ba","user_email":"zan@dataengineeracademy.com","user_name":"Zan Strmec"}],"event_type":"https://api.calendly.com/event_types/d639ecd3-8718-4068-955a-436b10d72c78","invitees_counter":{"total":1,"active":1,"limit":1},"location":{"location":null,"type":"custom"},"meeting_notes_html":null,"meeting_notes_plain":null,"name":"Data Engineer Academy Info

Session","start_time":"2025-07-09T17:45:00.000000Z","status":"active","updated_at":"2025-07-09T06:04:33.743275Z","uri":"https://api.calendly.com/scheduled_events/1ac9e88e-eae3-4e4b-b979-d770cff02d72"},"scheduling_method":null,"status":"active","text_reminder_number":null,"timezone":"Asia/Calcutta","tracking":{"utm_campaign":null,"utm_source":null,"utm_medium":null,"utm_content":null,"utm_term":null,"salesforce_uuid":null},"updated_at":"2025-07-09T06:04:33.761871Z","uri":"https://api.calendly.com/scheduled_events/1ac9e88e-eae3-4e4b-b979-d770cff02d72/invitees/22a0f2d6-1bde-4fc1-95c1-d969df1da21d"}}

2. **Example Flow: One Invitee Created**

    1. Webhook received with:
        - **event_type**: This will help you identify the marketing campaigns events. Use the below provided **event_types** to filter on the received data which involves the entire organization's data.
    2. You fetch:
        - **Event name**: The event name will help you identify the marketing events from the pool of all events.
    3. You extract:
        - **Webhook invite.created data** - Flatten the new events data in json format using webhook and prepare a table to join with the marketing data based on date**.**

3. **Support Data:** Below are the **event_types** for all three campaigns

    a. **facebook_paid_ads-** https://api.calendly.com/event_types/79652b8a-1688-4d0b-ba8f-733c70856de4
    b. **youtube_paid_ads-** https://api.calendly.com/event_types/2b5580c2-13e9-4283-8b4d-56401cf31554
    c. **tiktok_paid_ads-** https://api.calendly.com/event_types/70022cde-e64f-49a0-9362-a815186005dc

4. **Total Spends per Channel/Campaign[Data]:**

    You will receive data for total spends to produce the dashboard '**Cost Per Booking (CPB) by Channel'**. Data will be present in a public S3 bucket at **06:00 AM EST Daily** for **Day-1** spends**.**

    Data in the file ranges for a time period of 30 days at maximum. You can access the files dynamically with your ingestion logic by using this - **https://dea-data-bucket.s3.us-east-1.amazonaws.com/calendly_spend_data/spend_data_YYYY-MM-DD.json** where **YYYY-MM-DD[Day-1]** changes daily.

Also you can use this file - **https://dea-data-bucket.s3.us-east-1.amazonaws.com/calendly_spend_data/file_index.json** to read the file names and use it as per availability.

Data will be in json format & in **USD currency** which needs to be extracted into a table for your use.

Sample Data:


```
[ {
   "date": "2025-06-24",
   "channel": "facebook_paid_ads",
   "spend": 653.28
 },
 {
   "date": "2025-06-24",
   "channel": "youtube_paid_ads",
   "spend": 487.59
 },
 {
   "date": "2025-06-24",
   "channel": "tiktok_paid_ads",
   "spend": 345.12
 }]
```


Step 3: Build the Data Pipeline

Define and Calculate Metrics as per Business Requirement:

1.1 Daily Calls Booked by Source

- Goal: Count of Calendly bookings per source, per day

1.2 Analyze Cost Per Booking(CPB) by Channel

- **Goal**: Determine how much it costs to acquire a lead in each marketing channel.
- **Why it matters**: Identifies the most cost-efficient acquisition channels.
- **How to do it**:
    - Formula: Cost per Lead = Total Spend / Total Booked CAlls

    - Group by channel, then compute cost per lead.

1.3 Bookings Trend Over Time

- **Goal**: Track daily or weekly volume of Calendly bookings per source.
- **Why it matters**: Spot trends like campaign fatigue, seasonal changes, or the impact of new creatives.
- **How**: Group bookings by booking_date and source, then plot counts over time (line chart).

1.4 Channel Attribution

- **Goal**: Attribute bookings to specific marketing channels or campaigns using UTM parameters.
- **Why it matters**: Identifies which sources (Google, Facebook, YouTube, etc.) and campaigns generate the most and cheapest calls.
- **How**: Group bookings by source, and calculate:
    - total bookings
    - total spend
    - CPB

Then sort and visualize in a leaderboard or heatmap.

1.5 Booking Volume by Time Slot / Day of Week

- **Goal**: Understand when leads prefer booking calls.
- **Why it matters**: Helps with resource planning (e.g., setter/closer availability) and scheduling ad delivery (e.g., increasing bids near high-volume slots).
- **How**: Extract hour and day_of_week from Calendly booking timestamps and plot heatmaps or histograms grouped by time and channel.

1.6 Understand Meeting Load per Employee

- **Goal**: Calculate how many meetings each employee attends weekly.
- **Why it matters**: Identifies potential burnout or over-scheduling.
- **How to do it:**
    - Formula: Avg Meetings per Week = Total Meetings / Number of Weeks
    - Group by employee_id

Step 4: Build the Dashboard using Streamlit:

Please go through this link for working on Streamlit for Visualization: Streamlit Tutorial

1. **Task:**

Create **final reports or dashboards** which consolidate the **CRM**, **Wistia** & **Calendly** data to find relevance between the marketing efforts, video visitor analysis and the new lead generation.

2. **Calendly specific Deliverables:**

1.1 Daily Calls Booked by Source

- **Goal**: Count of Calendly bookings per source per day.
- **Why it matters: Monitors daily performance and compares lead generation efforts across channe**ls.
- **Visualizations**:
    - Line chart: Date vs Number of Bookings, color-coded by source
    - Optional: Daily comparison table by source
- Data Requirements:
    - booking_date, source, booking_id

1.2 Cost Per Booking (CPB) by Channel

- **Goal**: Calculate average cost per booking per channel.
- **Why it matters**: Reveals the most cost-effective channels for acquiring leads.
- **Formula**:
  CPB = Total Spend / Total Booked Calls
- **Visualizations:**
    - Bar chart: Channel vs CPB
    - KPI Tiles: Total Bookings, Total Spend, Average CPB
    - Table with sorting: Channel, Spend, Bookings, CPB
- **Data Requirements**:
    - channel, spend, booking_id

1.3 Bookings Trend Over Time

- **Goal**: Track daily/weekly booking volume by source.
- **Why it matters**: Helps identify dips, surges, or patterns due to campaigns, seasonality, or other factors.
- **Visualizations**:
  Line chart: Date vs Number of Bookings by source
    - Area chart: Cumulative Bookings
- **Data Requirements**:
    - booking_date, source, booking_id

1.4 Channel Attribution (CPB & Volume Leaderboard)

- **Goal**: Attribute bookings to channels using UTM parameters and rank by volume and cost.
- **Why it matters:** Allows precise ROI tracking by source.

- **Visualizations**:
    - Leaderboard table: Source | Total Bookings | Spend | CPB
    - Heatmap: CPB by Channel and Campaign
    - Bar chart: Top-performing sources by bookings
- **Data Requirements:**
    - source, campaign, booking_id, spend

1.5 Booking Volume by Time Slot / Day of Week

- **Goal**: Determine booking activity patterns across time slots and weekdays.
- **Why it matters**: Informs ad scheduling and team resource planning.
- **Visualizations**:
    - Heatmap: Hour of Day vs Day of Week (intensity = bookings)
    - Histogram: Bookings by Hour
    - Pie chart: Bookings by Day of Week
- **Data Requirements**:
    - booking_timestamp, source, booking_id

1.6 Understand Meeting Load per Employee

- **Goal**: Show how many meetings each employee attends per week.
- **Why it matters**: Helps track workload and prevent scheduling overload.
- **Formula**:
  Avg Meetings per Week = Total Meetings / Number of Weeks
- **Visualizations**:
    - Bar chart: Employee vs Avg Meetings/Week
    - KPI: Total Meetings, Max Meetings, Min Meetings
    - Optional line chart: weekly trend per employee
- **Data Requirements**:
    - employee_id, meeting_id, meeting_date

Submission and Review

1. **Task:**
    - Submit all deliverables, including:
        - Code files (ETL scripts).
        - Final Reports or dashboards.
        - Create a CI/CD pipeline using github.

2. **Instructions:**
    - Ensure all files are properly named and documented.
    - **Include a short video or presentation explaining the work**.